

Toy Card Shop Spec

Project Specifications

The project is to build a toy ordering form for a collectible trading card store.

Users want to create new orders, type in the name of the customer, add cards from a catalog to the order, change the quantities of cards, see the total price and average price of cards, and save the order.

Requirements

- When the web app “boots,” it has a blank state view that says “No Order Selected”
- A column on the left has links for “New Order” and a link for all existing orders
- The links to existing orders use the `order.orderName` property in the link text.
- When a new or existing order is opened, the main content view displays the following:
 - Editable Invoice field bound to `order.orderName`
 - Editable Customer field bound to `order.customerName`
 - A Total field showing the `order.totalPrice` using the \$ sign
 - A list of Items
 - The `item.description` in a textarea element
 - The Card Price field bound to `item.unitPrice`
 - The Qty numerical input bound to `item.quantity`
 - The Total as `Qty * Card Price`
 - A financial summary below the Items
 - # of Cards as the sum of all `item.quantity`
 - Average Card Price as `order total price / # of Cards`
 - Total order price (again)
 - At least one Card is displayed in a catalog to the right of the order details
 - The `card.cardName` is bolded (not editable)
 - The `card.description` displays (not editable)
 - The `card.catalogPrice` is bolded (not editable)
 - Users can click on an “Add to Order” button
 - A Save Changes button at the bottom of the page
- Order price totals are in sync with sum of all item prices
- Order and item data is saved to the npoint Orders API when Save Changes is clicked

Bonus Features

- Use the npoint Card API to pull multiple cards into the catalog view
- Correctness: make sure Average Card Price never throws a `DivideByZero` exception
- Routing: change the URL to include the `OrderID` of the current Order
- Relational data: ensure `orderIDs` and `itemIDs` are always unique
- UX: add the ability to remove items from an Order

- State: make sure order.TotalPrice is always the same as the sum of item prices
 - Even when new items were just added
 - Even when users change quantities of the items
 - Even when users remove items

Example Screenshots

The web app when first opened:

Trading Card Webstore

Welcome!

+ New Order

Order ORD-12345

No Order Selected

The web app when "New Order" is used:

Trading Card Webstore

Welcome!

+ New Order

Order ORD-12345

NEW ORDER

Invoice

Customer

Total \$0.00

Items

of Cards 0

Avg Card Price \$0.00

Total \$0.00

 Save Changes

Catalog

RAZZMATAZZ

A fantasy trading card of a wizard

\$5.00

+ Add To Order

BUBBA RUBY

A baseball trading card of a slugger

\$15.00

+ Add To Order

GOLDEN RING

A fantasy trading card of a famous magic item

The web app when an existing order is selected:

Trading Card Webstore

Welcome!

+ New Order

Order ORD-12345

ORD-12345

Invoice

Customer

Total \$40.00

Items

A fantasy trading card of a wizard

Card Price \$5.00

Qty

Total \$10.00

A baseball trading card of a slugger

Card Price \$15.00

Qty

Total \$30.00

of Cards 2

Avg Card Price \$20.00

Total \$40.00

 Save Changes

Catalog

RAZZMATAZZ
A fantasy trading card of a wizard
\$5.00 

BUBBA RUBY
A baseball trading card of a slugger
\$15.00 

GOLDEN RING
A fantasy trading card of a famous magic item
\$10.00 

JEREMY MCCOY
A baseball trading card of the legendary pitcher
\$25.00 

Example Test Cases

- User adds a new order
 - Sets Customer to “Joe Blue”
 - Adds “Golden Ring”
 - Sets quantity to 3
 - Order total should be \$30.00
 - Avg Card Price should be \$10.00
- User opens old order ORD-12345
 - User changes Item 1 (wizard card) quantity to 10
 - User changes Item 2 (slugger card) quantity to 1
 - Order total should be \$65.00
 - Avg Card Price should be \$5.91
- User adds a new order
 - Sets Invoice to O-9874
 - Sets Customer to “Jane Deer”
 - Adds “Golden Ring”
 - Sets quantity to 20
 - Adds another “Golden Ring”
 - Sets quantity to 0
 - Order total should be \$200.00
 - Avg Card Price should be \$10.00

Provided API, Typescript, and CSS

Orders API

GET <https://api.npoint.io/e3fd4314dba3baa26d90>

Example Response Body:

```
{
  "orders": [
    {
      "items": [
        {
          "cardID": 1,
          "itemID": 1,
          "orderID": 1,
          "quantity": 2,
          "unitPrice": 5,
          "description": "A fantasy trading card of a wizard"
        },
        {
          "cardID": 2,
          "itemID": 2,
          "orderID": 1,
          "quantity": 2,
          "unitPrice": 15,
          "description": "A baseball trading card of a slugger"
        }
      ],
      "orderID": 1,
      "cardCount": 2,
      "orderName": "ORD-12345",
      "totalPrice": 40,
      "customerName": "Joanna Colleen",
      "averageCardPrice": 20
    }
  ]
}
```

POST <https://api.npoint.io/e3fd4314dba3baa26d90>

Content-Type: application/json

Cards API

GET <https://api.npoint.io/24981ca2f9809bbef7d0>

Example Response Body:

```
{
  "cards": [
    {
      cardID: 1,
      cardName: 'RAZZMATAZZ',
      description: "A fantasy trading card of a wizard",
      catalogPrice: 5
    },
    {
      cardID: 2,
      cardName: 'BUBBA RUBY',
      description: "A baseball trading card of a slugger",
      catalogPrice: 15
    },
  ]
}
```

Typescript

```
/*
 * ITEM INTERFACE
 *
 * Items represent items in an ecommerce cart
 * They have a unit price and a quantity.
 * The unitPrice should be set from an ICard from the catalog.
 * The total price is not stored.
 */
export interface IItem {
  itemID: number;
  orderID: number;
  unitPrice: number;
  quantity: number;
  cardID: number;
  description: string;
}

/*
 * ORDER INTERFACE
 *
 * Orders are collections of items assigned to a customer
 * They have an arbitrary order name like 'ORD-5354'.
 * The totalPrice and averageCardPrice are computed from the items array.
 */
export interface IOrder {
  orderID: number;
  orderName: string;
  customerName: string;
  totalPrice: number;
  cardCount: number;
  averageCardPrice: number;
}
```

```

    items: IItem[];
}
/*
 * CARD INTERFACE
 *
 * Cards represent catalog items in an ecommerce online catalog.
 * They have a catalog price and a name/description.
 * The catalogPrice is used to fill the unitPrice of an IItem.
 */
export interface ICard {
    cardID: number;
    cardName: string;
    description: string;
    catalogPrice: number;
}
/
 * The ORDER API is the most important URL.
 * It allows GET and POST directly to this URL.
 *
 * The content of this endpoint follows the schema below:
 * {
 *   "orders": [
 *     {
 *       "orderId": 1,
 *       "items": [
 *         "itemID": 1,
 *         ...etc
 *       ],
 *       ...etc
 *     },
 *     {
 *       "orderId": 2,
 *       "items": [
 *         "itemID": 2,
 *         ...etc
 *       ],
 *       ...etc
 *     }
 *   ]
 * }
 *
 * POST requests should have Content-Type header set to 'application/json'
 */
export const ORDERS_API_URL = 'https://api.npoint.io/e3fd4314dba3baa26d90';

/*
 * The CARD API is a bonus API.
 * You will only need GET requests directly to this URL.
 *
 * The content of this endpoint follows the schema below:
 * {
 *   "cards": [
 *     {
 *       "cardID": 1,
 *       ...etc
 *     },
 *     {
 *       "cardID": 2,
 *       ...etc

```

```

*   }
* ]
*}
*/

export const CARDS_API_URL = 'https://api.npoint.io/24981ca2f9809bbef7d0';

/*
 * This is a shortcut to the most popular card, which MUST be displayed in the catalog view.
 *
 * Use this if you don't have time to use the CARDS_API_URL
 */
export const MOST_POPULAR_CARD: ICard = {
  cardID: 7,
  cardName: 'GOLDEN RING',
  description: 'A fantasy trading card of a famous magic item',
  catalogPrice: 10,
};

/**
 * This library function will create an IOrder in the correct schema
 */
export function GenerateNewOrder(
  lastOrderID: number,
  randomInvoiceNumber: string
): IOrder {
  return {
    orderID: lastOrderID++,
    orderName: randomInvoiceNumber,
    customerName: "",
    totalPrice: 0,
    cardCount: 0,
    averageCardPrice: 0,
    items: [],
  };
}

/**
 * This library function will create an IItem in the correct schema.
 *
 * Please note it requires an ICard to fill in the unitPrice, cardID, and description fields.
 */
export function GenerateNewItem(
  parentOrderID: number,
  fromCard: ICard,
  lastItemID: number
): IItem {
  return {
    itemID: lastItemID++,
    orderID: parentOrderID,
    unitPrice: fromCard.catalogPrice,
    quantity: 1,
    cardID: fromCard.cardID,
    description: `${fromCard.cardName}\n${fromCard.description}`,
  };
}

```

CSS

```
/* ## BODY AND TITLE STYLES ## */
body {
  font-family: sans-serif;
  padding: 0;
  margin: 0;
}
h1 {
  margin: 0;
  padding: 10px;
  background-color: #2b373d;
  color: #99ca3d;
}

/* ## LAYOUT UTILITIES ## */
.container {
  display: flex;
}
.column {
  flex-direction: column;
}

/* ## MAIN CONTENT VIEWS ## */
/* Use the empty view when no order is selected */
.cb-empty-view {
  padding: 5vh;
}

/* Use the order view when an order is selected */
.cb-order-view {
  padding: 5vh;
}

/* ## COMPONENT STYLING ## */
/* Button component styling */
.cb-button {
  background-color: #99ca3d;
  color: #333;
  font-weight: bold;
  padding: 5px 10px;
  cursor: pointer;
  border-radius: 6px;
}
.cb-button:hover {
  background-color: #a8de43;
}
.cb-button:active {
  background-color: #8ab637;
}
```

```
/* Navigation component styling */
.cb-nav {
  width: 200px;
  height: 95vh;
  display: block;
  background-color: #2b373d;
  color: #99ca3d;
}

/* Use cb-button inside the nav for actions and navigation links */
.cb-nav .cb-button {
  background-color: transparent;
  border-color: transparent;
  color: #99ca3d;
  width: 100%;
}

.cb-nav .cb-button:hover {
  background-color: #41525b;
}

/* Use cb-label inside the cb-nav for centered and green text */
.cb-nav .cb-label {
  color: #99ca3d;
  width: 100%;
  text-align: center;
  padding-top: 10px;
}

/* the item component is repeated inside the order view */
.cb-item-view {
  border: 1px solid black;
  border-radius: 6px;
  margin-bottom: 10px;
  padding: 2px 5px 5px 2px;
}

.cb-item-view textarea {
  width: 100%;
}

/* the catalog component is a sibling to the order view */
.cb-catalog {
  padding: 5vh;
}

/* the catalog card component is repeated inside the catalog view */
.cb-catalog-card {
  background-color: #eee;
  border: 1px solid black;
  border-radius: 0px;
  margin-bottom: 10px;
  padding: 2px 5px 5px 2px;
}

/* this nested style is for an "add card to order" button */
.cb-catalog-card .cb-button {
  float: right;
}
```